

Bst

二叉搜索树 & 平衡树

定义

二叉搜索树是一种二叉树的树形数据结构，其定义如下：

1. 空树是二叉搜索树。
2. 若二叉搜索树的左子树不为空，则其左子树上所有点的附加权值均小于其根节点的值。
3. 若二叉搜索树的右子树不为空，则其右子树上所有点的附加权值均大于其根节点的值。
4. 二叉搜索树的左右子树均为二叉搜索树。

二叉搜索树上的基本操作所花费的时间与这棵树的高度成正比。对于一个有 n 个结点的二叉搜索树中，这些操作的最优时间复杂度为 $O(\log n)$ ，最坏为 $O(n)$ 。随机构造这样一棵二叉搜索树的期望高度为 $O(\log n)$ 。

过程

二叉搜索树节点的定义

▼ 实现

```
1 struct TreeNode {
2     int key;
3     TreeNode* left;
4     TreeNode* right;
5     // 维护其他信息，如高度，节点数量等
6     int size;    // 当前节点为根的子树大小
7     int count;   // 当前节点的重复数量
8
9     TreeNode(int value)
10         : key(value), size(1), count(1), left(nullptr), right(nullptr) {}
11 };
```

遍历二叉搜索树

由二叉搜索树的递归定义可得，二叉搜索树的中序遍历权值的序列为非降的序列。时间复杂度为 $O(n)$ 。

遍历一棵二叉搜索树的代码如下：

▼ 实现

```
1 void inorderTraversal(TreeNode* root) {
2     if (root == nullptr) {
3         return;
4     }
5     inorderTraversal(root->left);
6     std::cout << root->key << " ";
7     inorderTraversal(root->right);
8 }
```

查找最小/最大值

由二叉搜索树的性质可得，二叉搜索树上的最小值为二叉搜索树左链的顶点，最大值为二叉搜索树右链的顶点。时间复杂度为 $O(n)$ 。

▼ 实现

```
1 int findMin(TreeNode* root) {
2     if (root == nullptr) {
3         return -1;
4     }
5     while (root->left != nullptr) {
6         root = root->left;
7     }
8     return root->key;
9 }
10
11 int findMax(TreeNode* root) {
12     if (root == nullptr) {
13         return -1;
14     }
15     while (root->right != nullptr) {
16         root = root->right;
17     }
18     return root->key;
19 }
```

搜索元素

在以 `root` 为根节点的二叉搜索树中搜索一个值为 `value` 的节点。

分类讨论如下：

- 若 `root` 为空，返回 `false`。
- 若 `root` 的权值等于 `value`，返回 `true`。
- 若 `root` 的权值大于 `value`，在 `root` 的左子树中继续搜索。
- 若 `root` 的权值小于 `value`，在 `root` 的右子树中继续搜索。

时间复杂度为 $O(n)$ 。

▼ 实现

```
1 bool search(TreeNode* root, int target) {
2     if (root == nullptr) {
3         return false;
4     }
5     if (root->key == target) {
6         return true;
7     } else if (target < root->key) {
8         return search(root->left, target);
9     } else {
10        return search(root->right, target);
11    }
12 }
```

插入，删除，修改都需要先在二叉搜索树中进行搜索。

插入一个元素

在以 `root` 为根节点的二叉搜索树中插入一个值为 `value` 的节点。

分类讨论如下：

- 若 `root` 为空，直接返回一个值为 `value` 的新节点。
- 若 `root` 的权值等于 `value`，该节点的附加域该值出现的次数自增。
- 若 `root` 的权值大于 `value`，在 `root` 的左子树中插入权值为 `value` 的节点。
- 若 `root` 的权值小于 `value`，在 `root` 的右子树中插入权值为 `value` 的节点。

时间复杂度为 。

▼ 实现

```
1 TreeNode* insert(TreeNode* root, int value) {
2     if (root == nullptr) {
3         return new TreeNode(value);
4     }
5     if (value < root->key) {
6         root->left = insert(root->left, value);
7     } else if (value > root->key) {
8         root->right = insert(root->right, value);
9     } else {
10        root->count++; // 节点值相等，增加重复数量
11    }
12    root->size = root->count + (root->left ? root->left->size : 0) +
13                (root->right ? root->right->size : 0); // 更新节点的子树大小
14    return root;
15 }
```

删除一个元素

在以 `root` 为根节点的二叉搜索树中删除一个值为 `value` 的节点。

先在二叉搜索树中搜索权值为 `value` 的节点，分类讨论如下：

- 若该节点的附加 `count` 大于 1，只需要减少 `count`。
- 若该节点的附加 `count` 为 1：
 - 若 `root` 为叶子节点，直接删除该节点即可。
 - 若 `root` 为链节点，即只有一个儿子的节点，返回这个儿子。
 - 若 `count` 有两个非空子节点，一般是用它左子树的最大值（左子树最右的节点）或右子树的最小值（右子树最左的节点）代替它，然后将它删除。

时间复杂度 。

▼ 实现

方法使用 `root = remove(root, 1)` 表示删除根节点为 `root` 树中值为 1 的节点，并返回新的根节点。

```

1 // 此处返回值为删除 value 后的新 root
2 TreeNode* remove(TreeNode* root, int value) {
3     if (root == nullptr) {
4         return root;
5     }
6     if (value < root->key) {
7         root->left = remove(root->left, value);
8     } else if (value > root->key) {
9         root->right = remove(root->right, value);
10    } else {
11        if (root->count > 1) {
12            root->count--; // 节点重复数量大于1, 减少重复数量
13        } else {
14            if (root->left == nullptr) {
15                TreeNode* temp = root->right;
16                delete root;
17                return temp;
18            } else if (root->right == nullptr) {
19                TreeNode* temp = root->left;
20                delete root;
21                return temp;
22            } else {
23                TreeNode* successor = findMinNode(root->right);
24                root->key = successor->key;
25                root->count = successor->count; // 更新重复数量
26                // 当 successor->count > 1时, 也应该删除该节点, 否则
27                // 后续的删除只会减少重复数量
28                successor->count = 1;
29                root->right = remove(root->right, successor->key);
30            }
31        }
32    }
33    // 继续维护size, 不写成 --root->size;
34    // 是因为value可能不在树中, 从而可能未发生删除
35    root->size = root->count + (root->left ? root->left->size : 0) +
36                (root->right ? root->right->size : 0);
37    return root;
38 }
39
40 // 此处以右子树的最小值为例
41 TreeNode* findMinNode(TreeNode* root) {
42     while (root->left != nullptr) {
43         root = root->left;
44     }
45     return root;
46 }

```

求元素的排名

排名定义为将数组元素升序排序后第一个相同元素之前的数的个数加一。

查找一个元素的排名, 首先从根节点跳到这个元素, 若向右跳, 答案加上左儿子节点个数加当前节点重复的数个数, 最后答案加上终点的左儿子子树大小加一。

时间复杂度 。

▼ 实现

```

1 int queryRank(TreeNode* root, int v) {
2     if (root == nullptr) return 0;
3     if (root->key == v) return (root->left ? root->left->size : 0) + 1;
4     if (root->key > v) return queryRank(root->left, v);
5     return queryRank(root->right, v) + (root->left ? root->left->size : 0) +
6         root->count;
7 }

```

查找排名为 k 的元素

在一棵子树中，根节点的排名取决于其左子树的大小。

- 若其左子树的大小大于等于，则该元素在左子树中；
- 若其左子树的大小在区间（count 为当前结点的值的出现次数）中，则该元素为子树的根节点；
- 若其左子树的大小小于，则该元素在右子树中。

时间复杂度。

▼ 实现

```

1 int querykth(TreeNode* root, int k) {
2     if (root == nullptr) return -1; // 或者根据需求返回其他合适的值
3     if (root->left) {
4         if (root->left->size >= k) return querykth(root->left, k);
5         if (root->left->size + root->count >= k) return root->key;
6     } else {
7         if (k == 1) return root->key;
8     }
9     return querykth(root->right,
10         k - (root->left ? root->left->size : 0) - root->count);
11 }

```

平衡树简介

使用搜索树的目的之一是缩短插入、删除、修改和查找（插入、删除、修改都包括查找操作）节点的时间。

关于查找效率，如果一棵树的高度为 h ，在最坏的情况，查找一个关键字需要对比 h 次，查找时间复杂度（也为平均查找长度 ASL, Average Search Length）不超过 h 。一棵理想的二叉搜索树所有操作的时间可以缩短到 $O(\log n)$ （ n 是节点总数）。

然而 $O(h)$ 的时间复杂度仅为理想情况。在最坏情况下，搜索树有可能退化为链表。想象一棵每个结点只有右孩子的二叉搜索树，那么它的性质就和链表一样，所有操作（增删改查）的时间是 $O(n)$ 。

可以发现操作的复杂度与树的高度 h 有关。由此引出了平衡树，通过一定操作维持树的高度（平衡性）来降低操作的复杂度。

平衡性的定义

关于一棵搜索树是否「平衡」，不同的平衡树中对「平衡」有着不同的定义。比如以 T 为根节点的二叉搜索树，左子树和右子树的高度相差很大，或者左子树的节点个数远大于右子树的节点个数，这棵树显然不具有平衡性。

对于二叉搜索树来说，常见的平衡性的定义是指：以 T 为根节点的树，每一个结点的左子树和右子树高度差最多为 1。

- [Splay 树](#) 中，对于任意节点的访问操作（搜索、插入还是删除），都会将被访问的节点移动到树的根节点位置。
- [AVL 树](#) 每个节点 N 维护以 N 为根节点的树的高度信息。AVL 树对平衡性的定义：如果 T 是一棵 AVL 树，当且

仅当左右子树也是 AVL 树，且 。

- [Size Balanced Tree](#) 每个节点 N 维护以 N 为根节点的树中节点个数 `size`。对平衡性的定义：任意节点的 `size` 不小于其兄弟节点（Sibling）的所有子节点（Nephew）的 `size`。
- [B 树](#) 对平衡性的定义：每个节点应该保持在一个预定义的范围内的关键字数量。

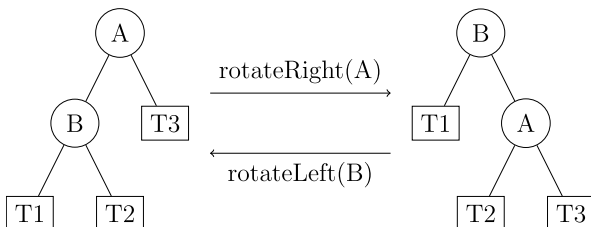
此外，对于拥有同样元素值集合的搜索树，平衡状态可能是不唯一的。也就是说，可能两棵不同的搜索树，含有的元素值集合相同，并且都是平衡的。

平衡的调整过程

对不满足平衡条件的搜索树进行调整操作，可以使不平衡的搜索树重新具有平衡性。

关于二叉平衡树，平衡的调整操作分为包括 **左旋（Left Rotate 或者 zag）** 和 **右旋（Right Rotate 或者 zig）** 两种。由于二叉平衡树在调整时需要保证中序遍历序列不变。这两种操作均不改变中序遍历序列。

在这里先介绍右旋，右旋也称为「右单旋转」或「LL 平衡旋转」。对于结点 A 的右旋操作是指：将 A 的左孩子 B 向右上旋转，代替 A 成为根节点，将 A 结点向右上旋转成为 B 的右子树的根结点，A 的原来的右子树变为 B 的左子树。



右旋操作只改变了三组结点关联，相当于对三组边进行循环置换一下，因此需要暂存一个结点再进行轮换更新。

对于右旋操作一般的更新顺序是：暂存 B 结点（新的根节点），让 A 的左孩子指向 B 的右子树，再让 A 的右孩子指针指向 B，最后让 A 的父结点指向暂存的 B。

完全同理，有对应的左旋操作，也称为「左单旋转」或「RR 平衡旋转」。左旋操作与右旋操作互为镜像。

下面给出左旋和右旋的代码。

▼ 实现

```
1 TreeNode* rotateLeft(TreeNode* root) {
2     TreeNode* newRoot = root->right;
3     root->right = newRoot->left;
4     newRoot->left = root;
5     // 更新相关节点的信息
6     updateHeight(root);
7     updateHeight(newRoot);
8     return newRoot; // 返回新的根节点
9 }
10
11 TreeNode* rotateRight(TreeNode* root) {
12     TreeNode* newRoot = root->left;
13     root->left = newRoot->right;
14     newRoot->right = root;
15     updateHeight(root);
16     updateHeight(newRoot);
17     return newRoot;
18 }
```

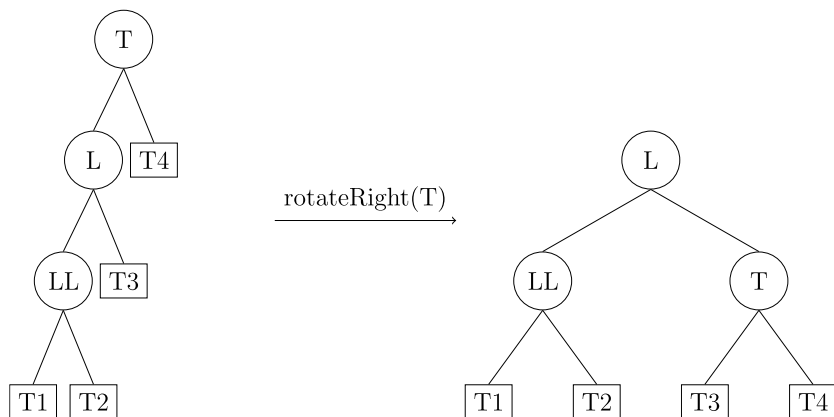
对于这段示例代码，在调用时需要保存 root 的父节点 pre。方法返回指向新的根节点的指针，只需要将 pre 指向新的根节点即可。

四种平衡性破坏的情况

虽然不同的二叉平衡树的定义有所区别，不同二叉平衡树区别只在于节点维护的信息不同，以及旋转调整后节点更新的信息不同。二叉平衡树平衡性被破坏的情况只有以下四种。进行平衡性调整的操作只包括左旋和右旋。以下先介绍四种情况，再对不同的二叉平衡树进行对比。

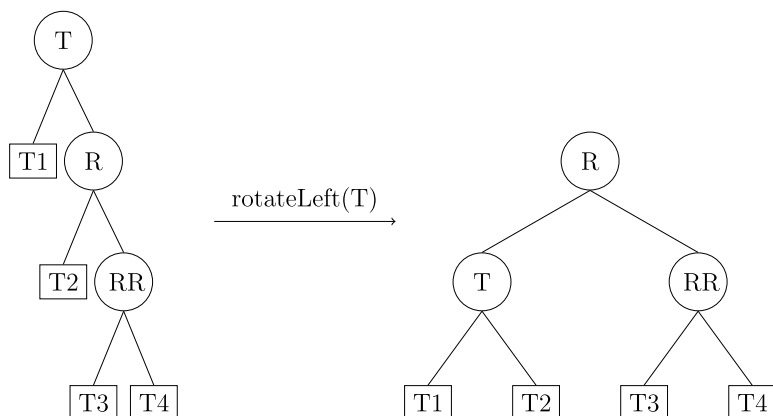
LL 型：T 的左孩子的左子树过长导致平衡性破坏。

调整方式：右旋节点 T。



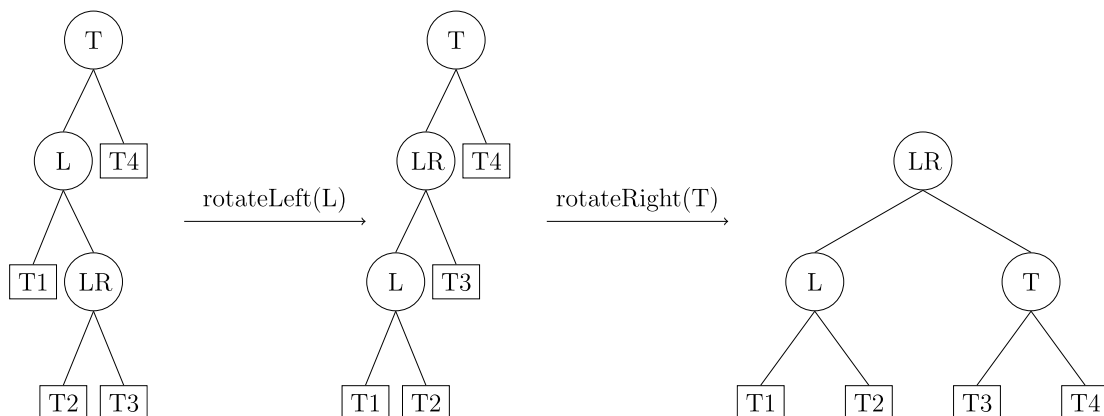
RR 型：与 LL 型类似，T 的右孩子的右子树过长导致平衡性破坏。

调整方式：左旋节点 T。



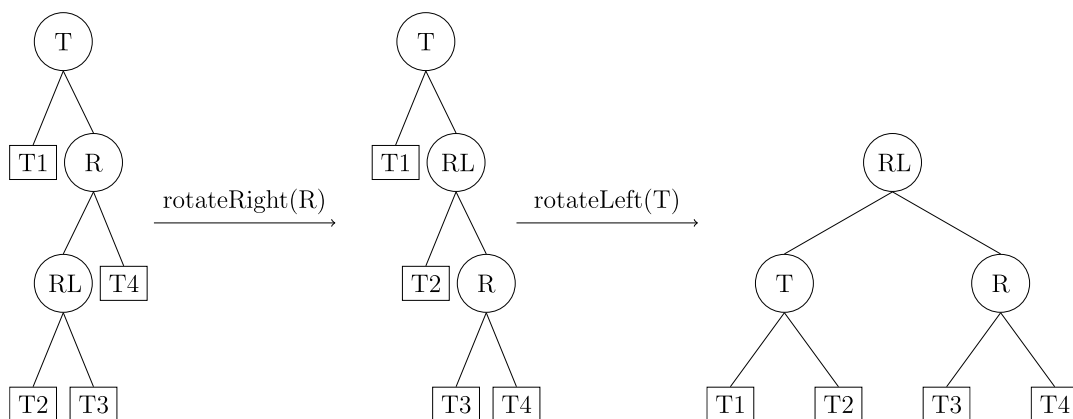
LR 型：T 的左孩子的右子树过长导致平衡性破坏。

调整方式：先左旋节点 L，成为 LL 型，再右旋节点 T。



RL 型：与 LR 型类似，T 的右孩子的左子树过长导致平衡性破坏。

调整方式：先右旋节点 R，成为 RR 型，再左旋节点 T。



本页面最近更新：2024/10/30 21:44:21, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [2323122](#), [aofall](#), [AtomAlpaca](#), [bililateral](#), [Bocity](#), [CoelacanthusHex](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [fearlessxjdx](#), [Great-designer](#), [H-J-Granger](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [ksyx](#), [linkerio](#), [Marcythm](#), [NachtgeistW](#), [ouuan](#), [Persdre](#), [q-wind](#), [shuzhouliu](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [wsyhb](#), [Yesphet](#), [yuhuoji](#), [HeRaNO](#), [Yushu-He](#), [zyzyyh](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用